Reliable transfer for a disaster-response network

CS3001 Computer Networks | Assignment II | Fall 2026

PROVISIONAL seed 0000: regenerate with your roll number before submission

Task 1: scenario and protocol choice

1. Why reliability matters. Rescue assignments must reach the correct team without changed coordinates or missing instructions. Casualty lists and supply requests must also arrive correctly and in order, because incomplete or repeated information can affect rescue decisions.

2. Why Stop-and-Wait wastes capacity. One 1,000-byte packet takes only 4 ms to transmit, but the satellite round trip takes 500 ms. The sender therefore spends most of each cycle waiting for an ACK instead of sending useful rescue data.

3. Why pipelining helps. A sender window allows several packets to be in flight while earlier ACKs are returning. This makes better use of the long satellite path and lets rescue updates leave the sender more quickly.

4. Protocol choice: SR. SR retransmits only an unacknowledged packet, so a single loss wastes less of the limited satellite bandwidth. Its receiver buffers valid out-of-order packets and delivers them when the gap is filled, whereas GBN discards them and sends them again. The extra receiver memory and per-packet timer bookkeeping are reasonable for a four-packet window.

Task 3: implementation and approach

The program uses a priority queue of events rather than real sleeping or sockets. Both protocols build packets with a 16-bit Internet checksum, send them through the same channel implementation, check received frames and recover through ACKs and timeouts. No reliable-transport library is used.

GBN: base identifies the oldest unacknowledged packet and nextseqnum identifies the next packet to send. The receiver accepts only expectedseqnum and sends the sequence number of its last in-order packet as a cumulative ACK. One logical timer tracks base; on expiry all outstanding packets are resent, and on progress the timer restarts for the new base.

SR: the sender remembers individual ACKs and maintains a timer for each unacknowledged packet. The receiver checks its window starting at rcv_base, buffers valid packets and delivers only a consecutive prefix. Duplicates in the previous receiver window are ACKed again, allowing recovery when an ACK is lost.

Assumptions and measurement rules

The link is full duplex at 2 Mbps per direction, with 250 ms one-way propagation. A data frame is exactly 1,000 bytes (993 bytes of application data and a 7-byte header/checksum); an ACK is 7 bytes. Transmission time and per-direction serialization are modeled. Loss, corruption and delay are independent draws per transmission, applied equally to data and ACKs; a dropped frame does not arrive.

The channel is FIFO in each direction: selected delays add 350 ms, and later arrivals wait behind a delayed frame. This queued-path assumption avoids arbitrary stale packets surviving sequence reuse; unlimited reordering would require a packet-lifetime rule. The timeout is 1 s. GBN requires N < 2^k and SR requires N <= 2^(k-1). Absolute counters are local bookkeeping only; receivers and ACK processing decide from modulo sequence numbers, never application packet identities.

Completion time is simulated seconds until the final required ACK reaches the sender. Transmitted packets count data attempts, including losses and retries; ACK attempts are reported separately in CSV/JSON. Reported throughput is 200 x 8,000 / completion time (useful data-frame bits/s, excluding retries and ACKs). Application-byte goodput would instead use 993 bytes per delivered packet.

Task 2: packet traces, N = 4

Assume immediate window refilling, no other faults, and a 1.5 s timeout for this trace, so ACKs of 3-5 return before packet 2 times out. ACKs below identify the received packet (not the next expected packet). S is the sender window; R is the SR receiver window; E is GBN expectedseqnum. Consecutive bursts are grouped; full send/receive/ACK event logs and the tables are included separately.

Go-Back-N: cumulative ACKs; receiver window size 1

Event / sender	Packet	Receiver	ACK	Window after event
1. Send initial burst	0,1,2,3	Deliver 0,1; lose 2; discard 3	Send 0,1,1	S[0-3]; E=2
2. Receive ACK0; send 4	4	In flight	Apply 0	S[1-4]; E=2
3. Receive ACK1; send 5	5	In flight	Apply 1	S[2-5]; E=2
4. Receive duplicate ACK1	-	Waiting for 2	Ignore 1	S[2-5]; E=2
5. 4 reaches receiver	4	Discard out of order	Send 1	S[2-5]; E=2
6. 5 reaches receiver	5	Discard out of order	Send 1	S[2-5]; E=2
7. Duplicate ACKs arrive	-	Waiting for 2	Ignore 1,1	S[2-5]; E=2
8. Timer for 2 expires	2,3,4,5	Resend all outstanding	-	S[2-5]; E=2
9. Retries reach receiver	2,3,4,5	Deliver 2,3,4,5 in order	Send 2,3,4,5	S[2-5]; E=6
10. Receive ACK2; send 6	6	In flight	Apply 2	S[3-6]; E=6
11. Receive ACK3; send 7	7	In flight	Apply 3	S[4-7]; E=6
12. Receive ACK4; send 8	8	In flight	Apply 4	S[5-8]; E=6
13. Receive ACK5; send 9	9	In flight	Apply 5	S[6-9]; E=6
14. Final burst arrives	6,7,8,9	Deliver 6,7,8,9	Send 6,7,8,9	S[6-9]; E=10
15. Receive ACK6	-	All delivered	Apply 6	S[7-9]; E=10
16. Receive ACK7	-	All delivered	Apply 7	S[8-9]; E=10
17. Receive ACK8	-	All delivered	Apply 8	S[9]; E=10
18. Receive ACK9	-	All delivered	Apply 9	S empty; E=10

Selective Repeat: individual ACKs; receiver buffers 3-5

Event / sender	Packet	Receiver	ACK	Window after event
1. Send initial burst	0,1,2,3	Deliver 0,1; lose 2; buffer 3	Send 0,1,3	S[0-3]; R[2-5]
2. Receive ACK0; send 4	4	In flight; buffer={3}	Mark 0	S[1-4]; R[2-5]
3. Receive ACK1; send 5	5	In flight; buffer={3}	Mark 1	S[2-5]; R[2-5]
4. Receive ACK3	-	Still waiting for 2	Mark 3	S[2-5]; R[2-5]
5. 4 reaches receiver	4	Buffer={3,4}; deliver nothing	Send 4	S[2-5]; R[2-5]
6. 5 reaches receiver	5	Buffer={3,4,5}; deliver nothing	Send 5	S[2-5]; R[2-5]
7. Receive ACK4 and ACK5	-	Buffer={3,4,5}	Mark 4,5	S[2-5]; R[2-5]
8. Timer for 2 expires	2	Resend only missing packet 2	-	S[2-5]; R[2-5]
9. Retry 2 reaches receiver	2	Deliver 2, then buffered 3,4,5	Send 2	S[2-5]; R[6-9]
10. ACK2; send next burst	6,7,8,9	In flight; buffer empty	Mark 2; base=6	S[6-9]; R[6-9]
11. 6 reaches receiver	6	Deliver 6	Send 6	S[6-9]; R[7-10]
12. 7 reaches receiver	7	Deliver 7	Send 7	S[6-9]; R[8-11]
13. 8 reaches receiver	8	Deliver 8	Send 8	S[6-9]; R[9-12]
14. 9 reaches receiver	9	Deliver 9	Send 9	S[6-9]; R[10-13]
15. Receive ACK6	-	All delivered	Mark 6	S[7-9]; R[10-13]
16. Receive ACK7	-	All delivered	Mark 7	S[8-9]; R[10-13]
17. Receive ACK8	-	All delivered	Mark 8	S[9]; R[10-13]
18. Receive ACK9	-	All delivered	Mark 9	S empty; R[10-13]

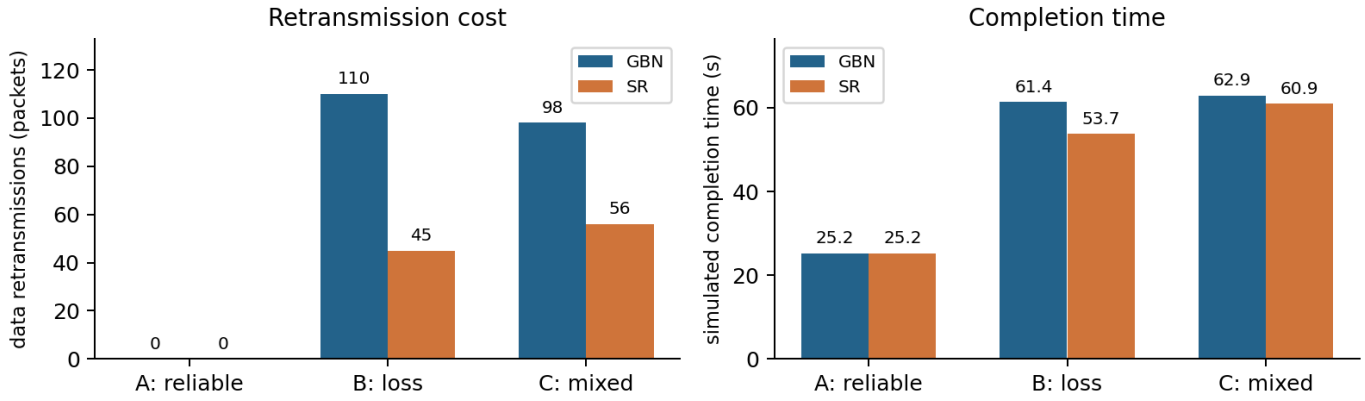
Retransmissions: GBN = 4 (2,3,4,5), SR = 1 (2). ACK0 and ACK1 open slots for 4 and 5 before the timeout. GBN discards 3-5 and resends the whole outstanding window; SR has already buffered and acknowledged 3-5. Sender windows near completion are clipped to the remaining packets; R ranges above 9 are logical positions, not extra data sent. Task 2 uses the given unique labels 0-9; executable traces use k=4, while Task 4 uses modulo-8 numbering.

Task 4: experiments and comparison

PROVISIONAL seed 0000: regenerate with your roll number before submission. Each protocol starts with the same seed and parameters: 200 packets, N=4, k=3, timeout=1 s. A: no faults. B: loss=0.10, corruption=0, delay=0. C: loss=0.05, corruption=0.05, delay=0.10. The RNG is reset for each run; protocols consume draws differently when their send histories differ.

Case	Protocol	Data sent	Retries	Delivered	Time (s)	Throughput (kbps)
A	GBN	200	0	200	25.213400	63.458
A	SR	200	0	200	25.213400	63.458
B	GBN	310	110	200	61.357848	26.077
B	SR	245	45	200	53.733260	29.777
C	GBN	298	98	200	62.919792	25.429
C	SR	256	56	200	60.941204	26.255

200 packets | N=4 | k=3 | PROVISIONAL seed 0000: regenerate with your roll number before submission



What the runs show

On the reliable channel, both protocols deliver all 200 packets without retransmissions and finish at the same simulated time. With faults, GBN can resend valid packets behind a gap because its receiver discards them, while SR keeps them and usually needs fewer retries. An ACK loss can still cause an SR retransmission even when the data was delivered, and delay can trigger a timeout if an ACK arrives too late. These are single seeded runs, not an average over many trials; their exact counts depend on the fault stream and channel assumptions.

Checks performed

The receiver output is compared byte for byte with the original ordered application messages at the end of every run. The tests also cover single-bit checksum errors in data and ACKs, clean-channel wraparound, the Task 2 retransmission counts, lost ACKs, loss at a sequence wrap, mixed faults across 15 seeds and three window sizes, GBN's largest legal window, invalid windows and total-loss failure. A configurable event limit prevents an unreachable channel from running forever.

How to reproduce

Create/activate a virtual environment, install requirements.txt, then run: `python prepare_submission.py --roll YOUR_ROLL_NUMBER`. The last four digits are extracted as the seed. This rebuilds all six runs, logs, CSV/JSON, trace tables, figures, this report and the submission ZIP. Use `python -m unittest -v` for the checks. The simulator itself uses only the Python standard library.

Task 5: transport-layer analysis

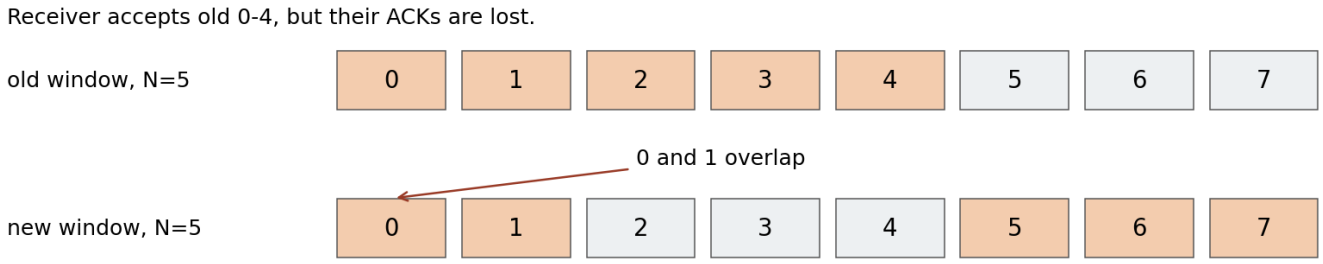
1. Pipelining and link utilization

L = 1,000 bytes x 8 = 8,000 bits; R = 2,000,000 bits/s; RTT = 0.500 s.
Transmission time L/R = 8,000 / 2,000,000 = **0.004 s = 4 ms**.
Stop-and-Wait: U = (L/R) / (RTT + L/R) = 0.004 / 0.504 = **0.0079365 = 0.79365%**.
With a four-packet pipeline: U = min(1, N(L/R)/(RTT + L/R))
= min(1, 4 x 0.004 / 0.504) = **0.031746 = 3.17460%**.

The four-packet window improves ideal utilization by a factor of four because it sends four packets per ACK cycle instead of one. Ideal useful frame throughput rises from 15.873 kbps to 63.492 kbps. These slide calculations ignore ACK serialization, errors and extra delays; the simulator includes a 28-microsecond ACK transmission time, so its clean throughput is slightly lower. Even N=4 leaves much of this high-delay link unused; approximately 126 packets in flight would be needed for ideal full utilization.

2. SR sequence numbers and safe window

(a) Three bits provide 2^3 = **8 sequence numbers: 0-7**.
(b) The maximum safe SR window is half the sequence space: N <= 2^(3-1) = **4 packets**.
(c) If N=5, suppose the receiver accepts old packets 0-4, but all five ACKs are lost. Its next receiver window becomes [5,6,7,0,1], while the sender is still waiting for ACKs for the original window. When old packet 0 is retransmitted, sequence number 0 is inside the new window, so the receiver cannot distinguish it from a new packet 0.



The old and new windows overlap at 0 and 1. For equal sender and receiver window sizes, 2N <= 8 prevents that overlap. As with the textbook protocols, the model also assumes old traffic cannot survive indefinitely across later reuse of the same sequence numbers.

Conclusion

SR is the better choice for this disaster-response link because it avoids resending valid out-of-order rescue updates. Both implementations preserve in-order, duplicate-free delivery under the stated channel model, while the checksums detect the injected corruption. The small window limits utilization even when the channel is reliable, so reliability and pipeline sizing both matter.

Submission and demonstration

Submit RollNo_A2.zip and a viewable Drive link to a short working video. Demonstrate a clean run, the forced packet-2 loss for both protocols, a mixed-fault run and a changed legal window size; explain one timeout, cumulative versus individual ACKs and SR buffering. Check the course portal for the deadline because the assignment sheet leaves it blank.